

Exercise 7 Solution

1. Assume the following two transactions start at nearly the same time and there is no other concurrent transaction. The 2nd operation of both transactions is Xlock(B). Is there a potential problem if Transaction 1 performs the operation first? What if Transaction 2 performs the operation first?

Transaction 1		Transaction 2	
1.1	Slock(A)	2.1	Slock(A)
1.2	Xlock(B)	2.2	Xlock(B)
1.3	Read(A)	2.3	Write(B)
1.4	Read(B)	2.4	Unlock(B)
1.5	Write(B)	2.5	Read(A)
1.6	Unlock(A)	2.6	Xlock(B)
1.7	Unlock(B)	2.7	Write(B)
		2.8	Unlock(A)
		2.9	Unlock(B)

Solution:

If Transaction 1 executes Step 1.2 before Transaction 2 executes Step 2.2, there would be no problem. This is because T1 releases the lock at the end. T2 has to wait till then. However, if Transaction 2 executes Step 2.2 first, Transaction 1 may have a dirty read of B if it tries to run Step 1.2 immediately after Transaction 2 acquires the Xlock on B. This is because when Transaction 2 releases the lock on B (Step 2.4), Transaction 1 will be granted the Xlock on B (Step 1.2). After that, Transaction 1 will read B (Step 1.4). After Transaction 1 completes, Transaction 2 will acquire another Xlock on B (Step 2.6) and then modify the object. In other words, the reading of B by Transaction 1 happens between two writes of B by Transaction 2.

2. What degree of isolation does the following transaction provide?

Slock(A)
 Xlock(B)
 Read(A)
 Write(B)
 Read(C)
 Unlock(A)
 Unlock(B)

Solution:

Degree 1 as there is one read operation 'Read(C)' without taking any lock. The only write operation has an exclusive lock associated with it. The transaction is two-phase with respect to exclusive lock.

3. The following operations are given with Degree 2 isolation locking principles in place. Convert the locking sequence to Degree 3.

Degree 2

Slock(A)
Read(A)
Unlock(A)
Xlock(C)
Xlock(B)
Write(B)
Slock(A)
Read(A)
Unlock(A)
Write(C)
Unlock(B)
Unlock(C)

Solution:

Degree 3

Slock(A)
Read(A)
Xlock(C)
Xlock(B)
Write(B)
Read(A)
Unlock(A)
Write(C)
Unlock(B)
Unlock(C)